



DA2 IoT Gateway -- Báo Cáo Tiến Độ Dự Án

1. Hệ Thống Cấu Hình Module Dựa Trên JSON (Đã Kiểm Thử)

1.1 Bối Cảnh

Trong phiên bản firmware trước, loại và hành vi hoạt động của module RF được cắm vào từng khe phần cứng đều được hardcode. Các lệnh AT khởi tạo, và chuỗi chân GPIO cho reset phần cứng đều được viết trực tiếp vào mã nguồn. Điều này khiến việc thay đổi loại module mà không cần biên dịch lại và nạp lại firmware là không thể. Đồng thời, việc hỗ trợ một module mới cũng đòi hỏi công sức kỹ thuật ở tầng firmware mỗi lần chọn module mới.

Mục tiêu của bản cập nhật này là đưa toàn bộ kiến thức đặc thù của module ra khỏi firmware và vào một file cấu hình có thể cập nhật tại runtime. Kết quả là hệ thống mà dự án gọi là Module Base Setting.

1.2 Cách Hệ Thống Hoạt Động

Cấu hình module được mô tả bằng định dạng JSON có cấu trúc. Mỗi file JSON mô tả một loại module và bao gồm tất cả thông tin firmware cần để vận hành module đó: loại bus truyền thông (UART, SPI, I2C hoặc USB), các tham số bus như tốc độ baud và parity, cùng danh sách đầy đủ các chức năng module hỗ trợ. Mỗi mục chức năng chứa chuỗi lệnh AT cần gửi, các chân GPIO cần điều khiển trước và sau lệnh, phản hồi mong đợi, và timeout tính bằng mili giây.

Ví dụ, một hàm reset phần cứng cho module BLE sẽ mô tả: kéo thấp chân GPIO reset trong 100ms, sau đó kéo cao lại và chờ 1 giây. Một hàm scan sẽ mô tả việc gửi chuỗi `AT+SCAN=5000` qua UART và chờ kết quả scan bất đồng bộ trả về. Các mô tả này nhất quán bất kể nhà cung cấp module BLE nào được sử dụng, miễn là tập lệnh AT khớp.

Sáu file cấu hình preset đã được chuẩn bị, bao gồm các module phổ biến nhất trong dự án: Zigbee E180-ZG120B, BLE STM32WB55, LoRa RAK3172, một biến thể BLE tùy chỉnh,

Zigbee STM32WB55, và LoRa Wio-E5. Một file ánh xạ tên `stack_id_map.json` liên kết các định danh khe được phát hiện bởi phần cứng với preset phù hợp.

1.3 Cài Đặt Firmware

Ở phía LAN MCU, một tầng Middleware chuyên dụng tên `JSON_Config_Parser` được viết để xử lý việc parse. Nó được chia thành các parser riêng biệt cho từng loại module, tất cả dùng chung một base cho các trường metadata. Kết quả parse được lưu vào các struct C có kiểu dữ liệu rõ ràng, chứa toàn bộ tham số chức năng trong bộ nhớ cho đến khi một handler task cần đến.

Một thành phần ứng dụng mới tên `Module_Config_Controller` nằm giữa parser và các driver giao tiếp phần cứng. Khi cấu hình parse cho biết module giao tiếp qua UART ở 115200 baud không có parity, controller này khởi tạo ngoại vi UART ESP32 tương ứng với đúng các thông số đó. Nếu cấu hình là SPI, nó cấu hình bus SPI tương ứng. Bản thân các handler task không bao giờ gọi trực tiếp các API phần cứng tầng thấp.

Ở tầng trên, `Module_Monitor_Task` quản lý vòng đời của tất cả handler task. Khi khởi động, nó đọc định danh khe phần cứng từ GPIO expander I2C, sau đó kiểm tra NVS flash để tìm cấu hình đã lưu trước đó cho từng khe. Nếu tìm thấy cấu hình và định danh module không thay đổi kể từ lần khởi động trước, nó tự động tải cấu hình và khởi động handler task tương ứng. Nếu không tìm thấy cấu hình, hệ thống chờ nhận từ ứng dụng PC qua liên kết SPI.

Khi cấu hình mới đến trong lúc runtime, monitor task parse JSON, lưu vào NVS, và khởi động lại handler task bị ảnh hưởng với cài đặt mới. Điều này có nghĩa là module có thể được đổi từ loại này sang loại khác tại runtime mà không cần nạp lại firmware.

1.4 Thay Đổi Trên Ứng Dụng PC

Các tab nâng cao trong ứng dụng PC (`DATN_config_app`) đã được thiết kế lại từ chỗ gửi lệnh AT trực tiếp sang xây dựng cấu hình JSON. Người dùng điền vào biểu mẫu với loại giao tiếp, tham số, và cài đặt từng chức năng. Ứng dụng tạo JSON đầy đủ trong panel xem trước bên phải màn hình, sau đó gửi đến gateway với tiền tố lệnh `CFBL:JSON:` cho BLE, `CFLR:JSON:` cho LoRa, và `CFZB:JSON:` cho Zigbee. Gateway phản hồi xác nhận sau khi cấu hình đã được parse và lưu.

Hành vi cũ là gửi từng lệnh AT thô từ giao diện đã được thay thế bằng mô hình trong đó

server hoặc ứng dụng PC gửi một chuỗi lệnh động như `AT+SCAN=5000`, và firmware tìm kiếm chức năng khớp trong JSON đã lưu. Nó tìm chức năng đúng bằng cách so khớp tiền tố (prefix matching), sau đó áp dụng chuỗi GPIO, delay và timeout từ cấu hình đã lưu. Cách tiếp cận này, gọi là prefix command matching, loại bỏ nhu cầu duy trì danh sách dài các case lệnh được hardcode trong firmware cho mọi biến thể module được hỗ trợ.

1.5 Kết Quả Kiểm Thử

Toàn bộ pipeline từ ứng dụng PC qua UART đến WAN MCU, qua SPI sang LAN MCU, qua parse JSON, lưu NVS, và khởi động lại handler task đã được kiểm thử và xác nhận hoạt động. Boot log xác nhận chuỗi phát hiện module và auto-restore đúng như mong đợi. JSON parser xử lý đúng tất cả các trường metadata, danh sách chức năng, mục điều khiển GPIO, và tham số giao tiếp cho cấu hình BLE. Tài liệu trong file `PARSER_VERIFICATION.md` ghi lại các bước và kết quả xác minh.

2. Cổng Cấu Hình Web Nhúng (Đã Kiểm Thử)

2.1 Bối Cảnh

Phiên bản trước của dự án yêu cầu cáp USB và ứng dụng desktop Python để cấu hình gateway. Mặc dù điều này chấp nhận được trong quá trình phát triển và kiểm thử, nhưng không thực tế khi triển khai thực tế. Kỹ thuật viên lắp đặt gateway tại công trình cần có khả năng cấu hình từ điện thoại hoặc laptop chỉ bằng mạng WiFi nội bộ, không cần cài đặt phần mềm.

Để giải quyết vấn đề này, một cổng cấu hình truy cập qua trình duyệt đã được thiết kế và nhúng trực tiếp vào firmware WAN MCU.

2.2 Quyết Định Kiến Trúc

Một trong những quyết định đầu tiên là nơi lưu trữ các file giao diện web. Cách tiếp cận phổ biến trên hệ thống nhúng là thêm một partition filesystem riêng vào flash và mount tại runtime. Tuy nhiên, điều này sẽ yêu cầu sửa đổi partition table, ảnh hưởng đến luồng cập nhật firmware over-the-air, và dẫn đến rủi ro mismatch phiên bản giữa web UI và firmware mà nó

điều khiển. Vì web UI gắn chặt với các cấu trúc dữ liệu của firmware, chúng cần thay đổi cùng nhau.

Cách tiếp cận được chọn là nhúng toàn bộ web UI như một binary resource được biên dịch trực tiếp vào firmware image sử dụng cơ chế `target_add_binary_data` của CMake. HTML, JavaScript và CSS được đóng gói vào một file duy nhất tại build time bằng công cụ `vite-plugin-singlefile`, inlines tất cả assets vào một file HTML tự chứa. Firmware serve file này từ một con trỏ `const` trong flash. Điều này giữ nguyên partition table và đảm bảo UI và firmware luôn đồng bộ với nhau.

2.3 Cài Đặt Backend

Backend web server được cài đặt bên trong component tên `Web_Config_Handler` trong dự án `DA2_esp`. Nó được tổ chức thành bốn file nguồn: khởi tạo và routing server, handler đọc/ghi cấu hình, báo cáo trạng thái, và captive DNS server.

Captive DNS server chịu trách nhiệm cho trải nghiệm lần khởi động đầu tiên. Khi gateway chưa có thông tin đăng nhập WiFi đã lưu, nó khởi động ở chế độ Access Point và tạo mạng WiFi tên `DA2-Gateway` theo sau bởi một định danh phần cứng ngắn. DNS server sau đó chặn mọi yêu cầu phân giải tên từ thiết bị kết nối và trả về địa chỉ IP của chính gateway. Điều này khiến hệ điều hành trên điện thoại hoặc laptop tự động hiển thị cổng cấu hình trong cửa sổ trình duyệt. Người dùng nhập SSID và mật khẩu WiFi, submit form, và gateway lưu thông tin đăng nhập rồi khởi động lại ở chế độ station.

Ở chế độ station, web server tiếp tục chạy trên địa chỉ IP nội bộ của gateway. Đăng ký mDNS làm cho cổng cấu hình có thể truy cập tại địa chỉ `gateway.local` mà không cần người dùng biết IP. Một chỉ báo trạng thái trên frontend poll endpoint trạng thái mỗi 5 giây để hiển thị gateway có đang kết nối Internet hay không, cùng với thông tin cường độ tín hiệu và phiên bản firmware.

Toàn bộ web server tích hợp vào firmware hiện tại như một nguồn lệnh mới. Khi người dùng submit thay đổi cấu hình qua trình duyệt, web handler đẩy cùng loại message vào cùng command queue mà UART handler và USB handler đã sử dụng. Điều này có nghĩa là file `config_handler.c` không cần thay đổi gì để hỗ trợ giao diện web.

2.4 Thiết Kế Frontend

Giao diện web phản chiếu bố cục của ứng dụng desktop Python. Nó bao gồm chế độ cơ bản để đọc trạng thái hệ thống và xem thông tin module, và chế độ nâng cao với các tab cho WiFi, LTE, MQTT, HTTP, CoAP, cấu hình module, và cập nhật firmware. Bảng màu và typography khớp với ứng dụng desktop để mang lại trải nghiệm nhất quán. Các tab nâng cao cho module RF tuân theo cách tiếp cận JSON configuration builder được giới thiệu trong bản cập nhật Module Base Setting.

2.5 Kết Quả Kiểm Thử

Cổng web đã được xác nhận trên phần cứng. Luồng captive portal khi lần đầu khởi động đã được kiểm thử trên cả Android và iOS, và trình duyệt đã chuyển hướng đúng đến trang cấu hình. Việc submit thông tin đăng nhập WiFi và kết nối lại ở chế độ station hoạt động đúng như mong đợi. Các thao tác đọc và ghi cấu hình qua trình duyệt đã được kiểm thử cho WiFi, LTE, và cài đặt server. Tính năng kích hoạt cập nhật over-the-air qua URL cũng đã được xác nhận hoạt động.

3. BLE Mesh Provisioner Gốc Trên ESP32-S3 (Chưa Kiểm Thử)

3.1 Bối Cảnh

Trong quá trình kiểm thử thử công bóng đèn LED thông minh Tuya E27 làm thiết bị demo cho gateway, một vấn đề tương thích căn bản đã được phát hiện. Tuya E27 quảng bá bản thân là một thiết bị BLE Mesh chưa được provision sử dụng kiểu advertisement non-connectable. Firmware lệnh AT trên module STM32WB55 yêu cầu một advertisement connectable tiêu chuẩn từ thiết bị đích để thiết lập kết nối GATT. Vì bóng đèn không bao giờ gửi advertisement connectable, lệnh `AT+CONNECT` sẽ không bao giờ thành công.

Giải pháp đúng duy nhất là cài đặt chức năng BLE Mesh provisioner. Một BLE Mesh provisioner giao tiếp với các node chưa được provision thông qua một thủ tục provisioning chuyên dụng, gán địa chỉ mạng và khóa bảo mật, sau đó điều khiển node bằng các thao tác Mesh model. ESP32-S3 trên LAN MCU hỗ trợ điều này gốc (natively) thông qua BLE Mesh

stack của ESP-IDF, vì vậy việc cài đặt được thực hiện ở đó mà không cần thay đổi gì trên WAN MCU.

3.2 Cài Đặt

Các file nguồn mới được thêm vào dưới component ứng dụng `BLE_Handler` hiện có. Việc khởi tạo provisioner đăng ký gateway như một mesh provisioner với network key, application key, và một tập client model binding. Các model được đăng ký bao gồm thao tác generic on/off, điều khiển độ sáng ánh sáng, và điều khiển nhiệt độ màu, khớp với khả năng của bóng đèn Tuya E27. Việc lựa chọn model được điều khiển bởi trường định danh model trong file JSON lệnh, vì vậy không có định danh model nào được hardcode trong firmware.

Một tiền tố lệnh mới `CFBN` được định nghĩa để định tuyến lệnh đến native BLE Mesh handler này, tuân theo đúng cùng mô hình như `CFBL` cho đường dẫn BLE AT command. Các lệnh bao gồm: quét tìm thiết bị chưa được provision, provision một thiết bị bằng UUID, gửi payload điều khiển đến các node đã được provision, và yêu cầu trạng thái từ các node riêng lẻ.

`Config_Handler` được mở rộng với hai loại message mới cho module này, và bảng frame type của `MCU_WAN_Handler` được cập nhật với định danh handler mới `BLN` để phân biệt traffic BLE Mesh gốc với traffic BLE AT command trên liên kết SPI giữa hai MCU.

`sdkconfig` cho LAN MCU cần các tùy chọn Kconfig cụ thể được bật để kích hoạt BLE Mesh stack, bao gồm vai trò provisioner, bearer provisioning PB-ADV, và các cài đặt client model cho các light control cluster.

3.3 Trạng Thái

Code đã được viết và đã được xác minh biên dịch không có lỗi. Kiểm thử phần cứng đang chờ thực hiện. Luồng khởi tạo provisioner và đường dẫn định tuyến lệnh đã được review ở cấp độ code, nhưng hành vi trên thiết bị thực với bóng đèn Tuya E27 thực tế chưa được quan sát. Công việc này được lên lịch cho giai đoạn kiểm thử tiếp theo.

4. Ứng Dụng Test Gateway: Điều Khiển Bóng Đèn Tuya E27 (Chưa Kiểm Thử)

4.1 Mục Đích

Để xác nhận toàn bộ hành vi end-to-end của gateway, một kịch bản test được thiết kế xung quanh việc điều khiển bóng đèn LED thông minh Tuya E27. Bóng đèn này được chọn vì nó hỗ trợ cả giao tiếp BLE và Zigbee, cho phép kiểm thử hai đường dẫn giao thức khác nhau qua gateway bằng cùng một thiết bị vật lý.

4.2 Nội Dung Đã Viết

Toàn bộ tập lệnh điều khiển bóng đèn qua gateway đã được thiết kế và ghi lại tài liệu cho cả đường dẫn BLE AT command qua module STM32WB55 và đường dẫn BLE Mesh gốc qua ESP32-S3. Đối với đường dẫn AT command, toàn bộ chuỗi từ reset phần cứng qua scan, connect, service discovery, bật notifications, và ghi payload điều khiển đèn đã được xây dựng và ghi tài liệu.

Tuya E27 sử dụng giao thức nhị phân độc quyền gửi qua BLE GATT. Mỗi payload lệnh bắt đầu bằng một header cố định theo sau là định danh data point, một byte type, một trường length, và giá trị thực tế. Các payload đã được tính toán và xác minh theo đặc tả Tuya DP cho điều khiển on/off, mức độ sáng, nhiệt độ màu, và chế độ màu RGB. Một chuỗi điều khiển Zigbee tương đương sử dụng định danh ZCL cluster cũng đã được ghi tài liệu cho đường dẫn Zigbee.

Các lệnh được tổ chức như các chỉ thị cấp gateway sử dụng tiền tố `CFBL` để ứng dụng server có thể kích hoạt từng hành động bằng một chuỗi duy nhất mà không cần biết chi tiết giao thức BLE bên trong. Gateway bóc tiền tố, áp dụng tham số GPIO và timing từ cấu hình JSON đã tải, và chuyển tiếp lệnh trần đến module vật lý.

Logic định tuyến lệnh của gateway đã được xác nhận qua code review để xử lý đúng hai lệnh GPIO-only đặc biệt `MODULE_HW_RESET` và `MODULE_WAKEUP`, không gửi gì qua UART mà thay vào đó toggle chân reset phần cứng hoặc chân wake trên GPIO expander.

4.3 Trạng Thái

Tập lệnh đã hoàn chỉnh và logic định tuyến đã được xác nhận qua code review. Bài test end-

to-end thực tế -- tức là bóng đèn LED thay đổi trạng thái vật lý để đáp lại lệnh phát ra từ ứng dụng server qua gateway -- chưa được thực hiện. Bài test này phụ thuộc vào việc có module BLE được kết nối và một bóng đèn Tuya E27 sẵn sàng để test.

5. Cập nhật firmware cho Board Phần Cứng Mới (Đang Tiến Hành)

5.1 Những Thay Đổi Trong Phiên Bản Board Mới

Sau khi hoàn thành các công việc phần mềm ở trên, nhóm phần cứng đã phát hành thiết kế board sửa đổi. Các thay đổi đủ lớn để yêu cầu cập nhật trên cả hai dự án firmware và nhiều tầng của software stack. Các thay đổi có tác động lớn nhất được tóm tắt dưới đây.

Trong board cũ, chip IO expander TCA6424A được gắn trực tiếp trên PCB chính. Chip này cung cấp 24 chân GPIO có thể điều khiển được, tổ chức thành ba port mỗi port 8 chân, và được dùng để điều khiển GPIO khe connector và quản lý nguồn. Board mới chuyển IO expander lên từng board adapter riêng lẻ, và đổi chip sang TCA6416A -- thiết bị nhỏ hơn 16 chân với chỉ hai port. Điều này ảnh hưởng đến mọi tầng firmware tương tác với tín hiệu GPIO qua expander.

Địa chỉ I2C của expander cũng thay đổi. Chip cũ luôn được cấu hình ở một địa chỉ cố định. Chip mới có thể được đặt ở một trong hai địa chỉ, và vì LAN MCU hiện làm việc với hai board adapter độc lập đồng thời, mỗi board phải có một địa chỉ khác nhau. Tài liệu cũng ghi nhận một ràng buộc thiết kế rằng cả hai adapter có thể vô tình có cùng địa chỉ nếu các board adapter không được sản xuất với cấu hình chân địa chỉ khác nhau -- điều này được đánh dấu là một rủi ro.

Một số chân GPIO cũng thay đổi trên phía LAN MCU. Chân UART receive và transmit của khe adapter thứ hai di chuyển sang số GPIO khác. Tất cả năm chân bus SPI cho giao tiếp adapter đều ở trên GPIO khác. Chân interrupt từ IO expander của mỗi adapter giờ có chân GPIO riêng trên LAN MCU thay vì dùng chung một chân. Một GPIO mới điều khiển USB bus switch định tuyến đường USB dùng chung đến một adapter hoặc adapter kia.

Tín hiệu điều khiển IO expander cũng di chuyển trên phía WAN MCU, và các chân điều khiển

nguồn và reset của modem LTE thay đổi trong bản đồ chân IO expander.

5.2 Các Công Việc Đã Hoàn Thành

Ba công việc trong kế hoạch thích nghi đã hoàn tất. Các chân giao tiếp SPI giữa WAN MCU và LAN MCU đã được xác nhận không thay đổi trong thiết kế mới, vì vậy không cần thay đổi firmware ở đây. Các chân interrupt, reset và data-ready cũng đã được xác nhận tương tự. Đường dẫn giao tiếp UART giữa hai MCU cũng đã được xác nhận không thay đổi.

5.3 Các Công Việc Còn Đang Tiến Hành

Thay đổi cơ bản nhất cần thực hiện là di chuyển TCA driver từ TCA6424A sang TCA6416A. Hai chip này tương tự về khái niệm nhưng có địa chỉ register khác nhau cho các register output và configuration. Mọi tầng firmware hiện tại gọi `TCA_PORT_2` sẽ thất bại vì port đó không tồn tại trên chip mới. Việc viết lại driver cũng phải được thiết kế theo instance thay vì singleton để cho phép hai chip expander độc lập cùng tồn tại.

Trên WAN MCU, stack handler cần được cập nhật để sử dụng enum GPIO 16 chân mới ánh xạ trực tiếp đến số port và chân vật lý trên TCA6416A. Quy ước đặt tên cũ từ `STACK_GPIO_PIN_1` đến `STACK_GPIO_PIN_11` cộng với tên `WAKE` và `PERST` riêng biệt đang được thay thế bằng một enumeration phẳng từ `P00` đến `P17` tương ứng trực tiếp với các chân expander. Một routine phát hiện phần cứng mới phải đọc bốn bit thấp của Port 0 trên expander để xác định địa chỉ phần cứng của module được cắm, thay thế cho pseudo-identifier hardcoded hiện tại.

Chức năng điều khiển modem LTE cũng phải được cập nhật. Firmware hiện tại coi chân 11 là tín hiệu điều khiển nguồn và chân 12 là tín hiệu điều khiển reset cho modem. Trong layout phần cứng mới, chân wake của modem ở `P05` và chân reset ở `P06`. Command parser cấu hình phải được cập nhật để nhận dạng nhãn chân mới, và cấu hình LTE đã lưu trong flash sẽ cần migration logic để xử lý các thiết bị vẫn còn số chân cũ được lưu từ trước khi cập nhật board.

Trên LAN MCU, stack handler cần một sửa đổi kiến trúc toàn diện. Thiết kế mới yêu cầu duy trì hai instance TCA driver riêng biệt, mỗi instance cho một khe adapter. Routine khởi tạo phải quét bus I2C, nhận dạng từng expander theo địa chỉ, đọc chân phát hiện khe để xác định mỗi expander thuộc khe vật lý nào, sau đó đọc bốn chân địa chỉ để xác định danh stack. Chỉ sau khi quá trình này hoàn tất, firmware mới biết module nào đang ở khe nào và tải cấu hình

phù hợp từ NVS.

Công việc còn cần thực hiện trên các chân UART LAN2 adapter, các chân bus SPI adapter, GPIO điều khiển USB switch, và một lượt xác minh các chân SD card để kiểm tra xem có chân nào xung đột với di chuyển chân UART mới hay không.

Handler nguồn điện trên WAN MCU, hiện điều khiển ba thanh rail nguồn qua TCA6424A Port 1, phải được thiết kế lại khi nhóm phần cứng làm rõ cách điều khiển rail nguồn được xử lý trong phiên bản board mới. Tương tự, cơ chế UART switch định tuyến UART của WAN MCU giữa module hiển thị và LAN MCU là tính năng mới trong thiết kế phần cứng chưa có cài đặt firmware tương ứng.
